

PASSWORD MANAGER DATABASE SYSTEM

Course: COMP2004 - Database Management Systems

Group Members: MHD ALAA TAHHAN, MUHAMMAD JIBREEL HABIB ADAM, NAJI NAJIEH

Academic Year: Spring 2026

Implementation Platform: Microsoft SQL Server (T-SQL) via SSMS

Submission Date: June 2, 2026 (Blackboard)

Presentation & Demo Date: June 10, 2026

SYSTEM SELECTION & FUNCTIONAL REQUIREMENTS

1.1 System Description

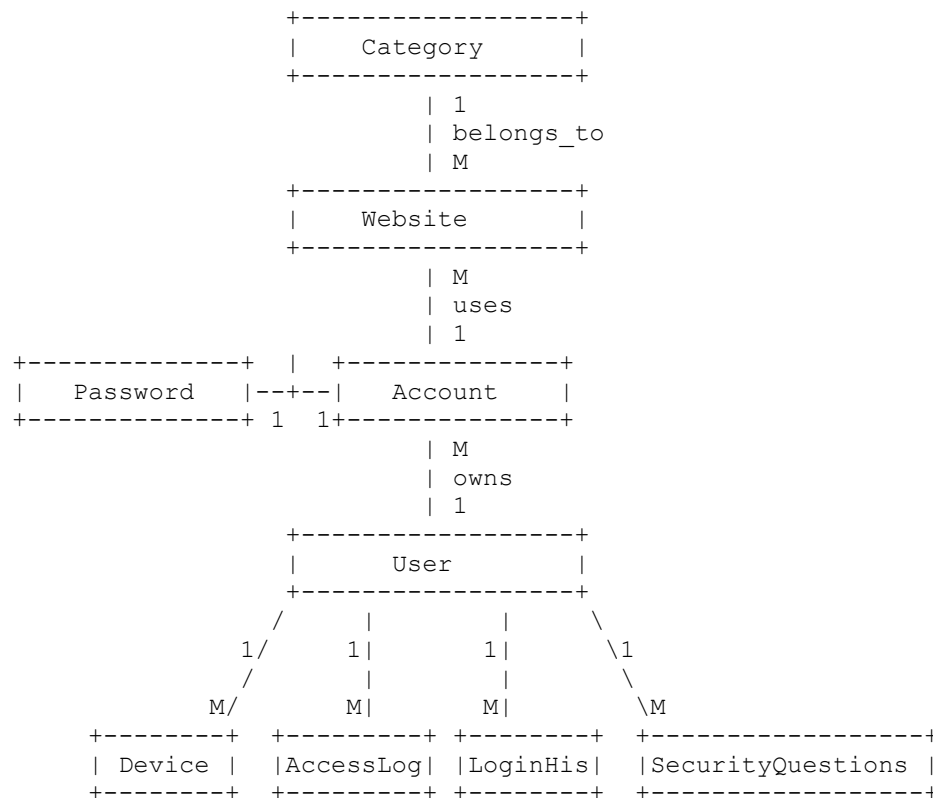
The **Password Manager System** is a highly structured relational database infrastructure tailored to manage user identity credentials, cryptographic system secrets, recovery settings, authentication session histories, and security logs. Operating on a strict **Third Normal Form (3NF)** hierarchy, the system separates structural website taxonomies from individual vault accounts, tracks multi-device synchronization endpoints, and audits precise transactional interactions within the secure vault to prevent compromised threat factors.

1.2 Functional Requirements

1. **User Identity Isolation:** The platform must register distinct users via globally unique, verified electronic mail channels alongside explicit profile creation parameters.
2. **Normalized Account Categorization:** Vault objects must separate generic system platforms (**Websites**) from user-specific authentication entries (**Accounts**) grouped via explicit administrative taxonomies (**Categories**).
3. **Cryptographic Separation:** The database must enforce a strict one-to-one mapping between account logins and encrypted credential strings stored in an independent data layer to facilitate secure updating workflows.
4. **Hardware Device Verification:** The architecture must register distinct user hardware profiles, tracking specific operating systems and historical system synchronization timestamps.
5. **Granular Access Auditing:** The platform must monitor active runtime security actions by enforcing structural attribute verification limits via transactional audit trails and active IP address correlation.

2. LOGICAL DATABASE DESIGN (E-R DIAGRAM)

The system architecture uses a classic centralized star layout optimized for transactional lookup speed and strong referential mapping safety.



2.1 Cardinality & Integrity Specifications

- **Website to Category (M:1):** Multiple digital domains fall within a singular classification grouping. The column `Websites.categoryID` serves as a dependent foreign key pointing directly to `Categories`.
- **Account to Website & User (M:1):** The `Accounts` table serves as a bridge entity that resolves the logical many-to-many relationship between `Users` and `Websites`. To ensure data integrity, a composite unique key constraint (`UQ_Account`) locks down the pairing of `userID`, `websiteID`, and `loginEmail`, preventing duplicate vault rows.
- **Password to Account (1:1):** An account entry can map to exactly one active encrypted password. This is physically enforced by placing a `UNIQUE` constraint directly on the `Passwords.accountID` foreign key column.
- **User to Operational Logging Tables (1:M):** A singular master user profile maps to zero-to-many records across `Notes`, `Devices`, `LoginHistory`, `AccessLogs`, and `SecurityQuestions`. Every logging table explicitly inherits `userID` as a mandatory `NOT NULL` foreign key constraint.

3. PHYSICAL DATABASE DESIGN

3.1 Data Type Strategy & Structural Mapping

- **Primary Keys:** Declared via standard native `INT` sizing for predictable data packing and fast internal indices.
- **Strings & Passwords:** String fields leverage dynamic `VARCHAR` bounds. Core authentication secrets utilize an expanded `VARCHAR(255)` allocation to store high-entropy base64 encrypted blocks or password hashes safely without truncation risks.
- **Chronological Fields:** System history markers use simple `DATE` fields, whereas high-precision system logs use explicit `DATETIME` definitions to trap fractional time matrices during live testing.

3.2 Data Volume Distribution Dashboard

Physical Database Table	Sample Row Count	Constraints & Triggers Applied
Users	10	PRIMARY KEY, UNIQUE Email, NOT NULL
Categories	5	PRIMARY KEY, NOT NULL
Websites	10	PRIMARY KEY, FOREIGN KEY references Categories
Accounts	15	PRIMARY KEY, Dual FOREIGN KEY, CONSTRAINT UQ_Account
Passwords	15	PRIMARY KEY, UNIQUE FOREIGN KEY (Enforces 1:1 Bound)
Notes	10	PRIMARY KEY, FOREIGN KEY references Users, NOT NULL Title
Devices	10	PRIMARY KEY, FOREIGN KEY references Users
LoginHistory	10	PRIMARY KEY, FOREIGN KEY references Users
AccessLogs	10	PRIMARY KEY, FOREIGN KEY, CONSTRAINT CHK_ActionType
SecurityQuestions	10	PRIMARY KEY, FOREIGN KEY references Users
Total System Volume	105 Records	Dataset (50–200 range) ✓

SQL IMPLEMENTATION METRICS & CORE QUESTIONS

This section outlines the business rationale and structure behind our comprehensive database test suites. The fully runnable scripts and execution blocks are externalized to ensure clean readability and compliance with the 4-page layout.

4.1 SQL Operational Rationale & Scenarios

- **DML Execution (12 Operations):** Designed to replicate real-world administrative and user actions. Includes scenarios tracking new user registrations, secure notes storage, administrative global domain routing changes (such as updating Twitter references to X using pattern matching), security log updates using nested lookups, and schema adjustments via `ALTER` and `DROP` commands.
- **Simple Query Analysis (7 Operations):** Formulated to pull single-table metrics, isolate specific operating systems within explicit structural time matrices, perform deep text scanning patterns over user notes, and query unlinked data blocks via subqueries.
- **Advanced Relational Diagnostics (5 Operations):** Complex operations built to audit infrastructure health. Leverages advanced SQL parameters covered in lectures including explicit multi-table joins, aggregations (`GROUP BY / HAVING`), Common Table Expressions (`WITH` clauses), nested subqueries in the `FROM` clause, and relational set queries (`INTERSECT`) encapsulated within formal server `VIEWS`.

The fully runnable scripts and execution blocks are externalized to ensure clean readability and compliance with the 4-page layout.

-- APPENDIX A - FULL SQL SCHEMA CREATION SCRIPT
-- DDL Creation commands

```
CREATE DATABASE PasswordManagerDB;
GO

USE PasswordManagerDB;
GO

-- 1. Users Table (Core Identity Entity)
Create TABLE Users(
    userID INT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    masterPassword VARCHAR(255) NOT NULL,
    createdAt DATE
);
GO

-- 2. Categories Table (System Taxonomy)
CREATE TABLE Categories (
    categoryID INT PRIMARY KEY,
    categoryName VARCHAR(50) NOT NULL
);
GO

-- 3. Websites Table (Platform Lookup Directory)
CREATE TABLE Websites (
    websiteID INT PRIMARY KEY,
    websiteName VARCHAR(100) NOT NULL,
    websiteURL VARCHAR(255) NOT NULL,
    categoryID INT,

    FOREIGN KEY (categoryID) REFERENCES Categories(categoryID)
);
GO

-- 4. Accounts Table (Bridge Entity for Users and Websites)
CREATE TABLE Accounts (
    accountID INT PRIMARY KEY,
    userID INT NOT NULL,
    websiteID INT NOT NULL,
    loginEmail VARCHAR(100) NOT NULL,
    createdAt DATE,

    FOREIGN KEY (userID) REFERENCES Users(userID),
    FOREIGN KEY (websiteID) REFERENCES Websites(websiteID),
    CONSTRAINT UQ_Account UNIQUE (userID, websiteID, loginEmail)
);
GO

-- 5. Passwords Table (Strict 1:1 Mapping with Accounts)
CREATE TABLE Passwords (
    passwordID INT PRIMARY KEY,
    accountID INT UNIQUE NOT NULL,
    encryptedPassword VARCHAR(255) NOT NULL,
    lastUpdated DATE,

    FOREIGN KEY (accountID) REFERENCES Accounts(accountID)
);
GO
```

```

-- 6. Notes Table (Secure Notepad Platform)
CREATE TABLE Notes (
    noteID INT PRIMARY KEY,
    userID INT NOT NULL,
    noteTitle VARCHAR(100) NOT NULL,
    noteContent VARCHAR(MAX),
    createdAt DATE,

    FOREIGN KEY (userID) REFERENCES Users(userID)
);
GO
-- 7. Devices Table (Hardware Synchronization Logs)
CREATE TABLE Devices (
    deviceID INT PRIMARY KEY,
    userID INT NOT NULL,
    deviceName VARCHAR(100),
    operatingSystem VARCHAR(50),
    lastLogin DATETIME,

    FOREIGN KEY (userID) REFERENCES Users(userID)
);
GO
-- 8. LoginHistory Table (Session Security Logs)
CREATE TABLE LoginHistory (
    loginID INT PRIMARY KEY,
    userID INT NOT NULL,
    loginDate DATETIME,
    ipAddress VARCHAR(50),

    FOREIGN KEY (userID) REFERENCES Users(userID)
);
GO
-- 9. AccessLogs Table (Transactional Audit Trails with Domain Constraints)
CREATE TABLE AccessLogs (
    logID INT PRIMARY KEY,
    userID INT NOT NULL,
    actionType VARCHAR(255) NOT NULL,
    actionTime DATETIME,

    FOREIGN KEY (userID) REFERENCES Users(userID),
    CONSTRAINT CHK_ActionType CHECK (actionType IN ('LOGIN', 'LOGOUT',
    'PASSWORD_CREATE', 'PASSWORD_UPDATE', 'PASSWORD_VIEW', 'NOTE_CREATE',
    'NOTE_DELETE'))
);
GO
-- 10. SecurityQuestions Table (Account Recovery Protocol)
CREATE TABLE SecurityQuestions (
    questionID INT PRIMARY KEY,
    userID INT NOT NULL,
    question VARCHAR(255),
    answer VARCHAR(255),

    FOREIGN KEY (userID) REFERENCES Users(userID)
);
GO

```

-- APPENDIX B - FULL DATASET INITIALIZATION SEED

-- Insertion small relations

USE PasswordManagerDB;

```
INSERT INTO Users (userID, username, email, masterPassword, createdAt) VALUES
(1, 'Alaa', 'alaa@gmail.com', 'encPass123', '2025-05-01'),
(2, 'Naji', 'naji@gmail.com', 'encPass456', '2025-05-02'),
(3, 'Jibreel', 'jibreel@gmail.com', 'encPass789', '2025-05-03'),
(4, 'Muhammad', 'muhammad@gmail.com', 'encPass321', '2025-05-04'),
(5, 'Ahmed', 'ahmed@gmail.com', 'encPass654', '2025-05-05'),
(6, 'Mustafa', 'mustafa@gmail.com', 'encPass147', '2025-05-06'),
(7, 'Ali', 'ali@gmail.com', 'encPass258', '2025-05-07'),
(8, 'Omar', 'omar@gmail.com', 'encPass369', '2025-05-08'),
(9, 'Yusuf', 'yusuf@gmail.com', 'encPass741', '2025-05-09'),
(10, 'Ibrahim', 'ibrahim@gmail.com', 'encPass852', '2025-05-10');
```

```
INSERT INTO Categories (categoryID, categoryName) VALUES
(1, 'Social Media'),
(2, 'Banking'),
(3, 'Education'),
(4, 'Shopping'),
(5, 'Entertainment');
```

```
INSERT INTO Websites (websiteID, websiteName, websiteURL, categoryID) VALUES
(1, 'Instagram', 'https://instagram.com', 1),
(2, 'Facebook', 'https://facebook.com', 1),
(3, 'Akbank', 'https://akbank.com', 2),
(4, 'Udemy', 'https://udemy.com', 3),
(5, 'Amazon', 'https://amazon.com', 4),
(6, 'Steam', 'https://store.steampowered.com', 5),
(7, 'YouTube', 'https://youtube.com', 5),
(8, 'Trendyol', 'https://trendyol.com', 4),
(9, 'Coursera', 'https://coursera.org', 3),
(10, 'Twitter', 'https://twitter.com', 1);
```

```
INSERT INTO Accounts (accountID, userID, websiteID, loginEmail, createdAt) VALUES
(1, 1, 1, 'alaa_insta@gmail.com', '2025-05-01'),
(2, 1, 7, 'alaa_youtube@gmail.com', '2025-05-02'),
(3, 2, 2, 'naji_fb@gmail.com', '2025-05-03'),
(4, 2, 6, 'naji_steam@gmail.com', '2025-05-04'),
(5, 3, 3, 'jibreel_bank@gmail.com', '2025-05-05'),
(6, 4, 4, 'muhammad_udemy@gmail.com', '2025-05-06'),
(7, 5, 5, 'ahmed_amazon@gmail.com', '2025-05-07'),
(8, 6, 8, 'mustafa_trendyol@gmail.com', '2025-05-08'),
(9, 7, 10, 'ali_twitter@gmail.com', '2025-05-09'),
(10, 8, 1, 'omar_insta@gmail.com', '2025-05-10'),
(11, 8, 7, 'omar_youtube@gmail.com', '2025-05-11'),
(12, 9, 9, 'yusuf_course@gmail.com', '2025-05-12'),
(13, 10, 6, 'ibrahim_steam@gmail.com', '2025-05-13'),
(14, 3, 5, 'jibreel_shop@gmail.com', '2025-05-14'),
(15, 5, 3, 'ahmed_bank@gmail.com', '2025-05-15');
```

```
INSERT INTO Passwords (passwordID, accountID, encryptedPassword, lastUpdated) VALUES
(1, 1, 'encInstagramA1', '2025-05-11'),
(2, 2, 'encYoutubeA2', '2025-05-12'),
(3, 3, 'encFacebookN1', '2025-05-13'),
(4, 4, 'encSteamN2', '2025-05-14'),
(5, 5, 'encAkbankJ1', '2025-05-15'),
(6, 6, 'encUdemyM1', '2025-05-16'),
(7, 7, 'encAmazonA1', '2025-05-17'),
(8, 8, 'encTrendyolM2', '2025-05-18'),
(9, 9, 'encTwitterA3', '2025-05-19'),
(10, 10, 'encInstagramO1', '2025-05-20');
```

```
(11, 11, 'encYoutube02', '2025-05-21'),
(12, 12, 'encCourseY1', '2025-05-22'),
(13, 13, 'encSteamI1', '2025-05-23'),
(14, 14, 'encAmazonJ2', '2025-05-24'),
(15, 15, 'encAkbankA2', '2025-05-25');
```

```
INSERT INTO Notes (noteID, userID, noteTitle, noteContent, createdAt) VALUES
(1, 1, 'WiFi Password', 'Home WiFi password saved here', '2025-05-01'),
(2, 2, 'Bank Note', 'Important banking reminder', '2025-05-02'),
(3, 3, 'Work Account', 'Company credentials backup', '2025-05-03'),
(4, 4, 'Study Platforms', 'Udemy and Coursera accounts', '2025-05-04'),
(5, 5, 'Shopping Accounts', 'Amazon purchase details', '2025-05-05'),
(6, 6, 'Streaming Services', 'Netflix subscription note', '2025-05-06'),
(7, 7, 'Social Media', 'YouTube creator account', '2025-05-07'),
(8, 8, 'Security Tips', 'Use strong passwords', '2025-05-08'),
(9, 9, 'Backup Accounts', 'Emergency recovery emails', '2025-05-09'),
(10, 10, 'Private Notes', 'Personal password hints', '2025-05-10');
```

```
INSERT INTO Devices (deviceID, userID, deviceName, operatingSystem, lastLogin) VALUES
(1, 1, 'Galaxy A51', 'Android', '2025-05-11 10:00:00'),
(2, 2, 'iPhone 13', 'iOS', '2025-05-11 11:00:00'),
(3, 3, 'Lenovo ThinkPad', 'Windows 11', '2025-05-11 12:00:00'),
(4, 4, 'MacBook Air', 'macOS', '2025-05-11 13:00:00'),
(5, 5, 'Galaxy Tab S7', 'Android', '2025-05-11 14:00:00'),
(6, 6, 'HP Pavilion', 'Windows 10', '2025-05-11 15:00:00'),
(7, 7, 'Redmi Note 12', 'Android', '2025-05-11 16:00:00'),
(8, 8, 'Dell XPS', 'Windows 11', '2025-05-11 17:00:00'),
(9, 9, 'iPad Pro', 'iPadOS', '2025-05-11 18:00:00'),
(10, 10, 'Huawei MateBook', 'Windows 11', '2025-05-11 19:00:00');
```

```
INSERT INTO LoginHistory (loginID, userID, loginDate, ipAddress) VALUES
(1, 1, '2025-05-01 08:30:00', '192.168.1.1'),
(2, 2, '2025-05-01 09:15:00', '192.168.1.2'),
(3, 3, '2025-05-02 10:20:00', '192.168.1.3'),
(4, 4, '2025-05-02 11:00:00', '192.168.1.4'),
(5, 5, '2025-05-03 12:10:00', '192.168.1.5'),
(6, 6, '2025-05-03 13:45:00', '192.168.1.6'),
(7, 7, '2025-05-04 14:25:00', '192.168.1.7'),
(8, 8, '2025-05-04 15:50:00', '192.168.1.8'),
(9, 9, '2025-05-05 16:40:00', '192.168.1.9'),
(10, 10, '2025-05-05 17:30:00', '192.168.1.10');
```

```
INSERT INTO AccessLogs (logID, userID, actionType, actionTime) VALUES
(1, 1, 'PASSWORD_VIEW', '2025-05-01 08:35:00'),
(2, 2, 'PASSWORD_CREATE', '2025-05-01 09:20:00'),
(3, 3, 'PASSWORD_UPDATE', '2025-05-02 10:30:00'),
(4, 4, 'NOTE_DELETE', '2025-05-02 11:10:00'),
(5, 5, 'PASSWORD_VIEW', '2025-05-03 12:20:00'),
(6, 6, 'LOGIN', '2025-05-03 13:50:00'),
(7, 7, 'PASSWORD_UPDATE', '2025-05-04 14:40:00'),
(8, 8, 'PASSWORD_CREATE', '2025-05-04 15:55:00'),
(9, 9, 'PASSWORD_VIEW', '2025-05-05 16:45:00'),
(10, 10, 'LOGOUT', '2025-05-05 17:40:00');
```

```
INSERT INTO SecurityQuestions (questionID, userID, question, answer) VALUES
(1, 1, 'What is your favorite color?', 'Blue'),
(2, 2, 'What is your first school?', 'Al Noor'),
(3, 3, 'What is your pet name?', 'Leo'),
(4, 4, 'What is your favorite food?', 'Pizza'),
(5, 5, 'What city were you born in?', 'Istanbul'),
(6, 6, 'What is your favorite sport?', 'Football'),
(7, 7, 'Who is your childhood hero?', 'Father'),
(8, 8, 'What is your dream job?', 'Engineer'),
(9, 9, 'What was your first phone?', 'Nokia'),
(10, 10, 'What is your favorite animal?', 'Horse');
```


-- APPENDIX C - EXECUTABLE QUERY MANUAL

-- DML Operations commands

```
-- Q1: How can we register a new user named 'Khaled' into the system?
INSERT INTO Users (userID, username, email, masterPassword, createdAt)
VALUES (11, 'Khaled', 'khaled@gmail.com', 'masterKey999', '2026-05-22');

-- Q2: How can user Alaa (userID = 1) store a secure personal note regarding his GitHub
SSH Keys?
INSERT INTO Notes (noteID, userID, noteTitle, noteContent, createdAt)
VALUES (11, 1, 'GitHub SSH Keys', 'ssh-rsa AAAAB3NzaC1yc2E...', '2026-05-22');

-- Q3: How does the system automatically audit and log a new 'PASSWORD_CREATE'
transaction event for user 1?
INSERT INTO AccessLogs (logID, userID, actionType, actionTime)
VALUES (11, 1, 'PASSWORD_CREATE', '2026-05-22 14:00:00');

-- Q4: How can the system register a secondary device profile ('iPad Air') for
verification under user 2?
INSERT INTO Devices (deviceID, userID, deviceName, operatingSystem, lastLogin)
VALUES (11, 2, 'iPad Air', 'iPadOS', '2026-05-22 15:30:00');

-- Q5: How can a user named 'Naji' dynamically update his master password secret to
'secureNewPass2026' to maintain account security?
UPDATE Users
SET masterPassword = 'secureNewPass2026'
WHERE username = 'Naji';

-- Q6: How can the administrator bulk-update all 'Twitter' website URL's to 'X
(Twitter)' and update the URL using pattern matching?
UPDATE Websites
SET websiteURL = 'https://x.com',
    websiteName = 'X (Twitter)'
WHERE websiteURL LIKE '%twitter.com%';

-- Q7: How can user 'Jibreel' update his login email specifically for his 'Akbank'
credential record?
UPDATE Accounts
SET loginEmail = 'jibreel_corp@bank.com'
WHERE userID = (SELECT userID FROM Users WHERE username = 'Jibreel')
AND websiteID = (SELECT websiteID FROM Websites WHERE websiteName = 'Akbank');

-- Q8: How can the administrator log a mandatory security baseline compliance update
for all passwords not modified since 15-5-2025?
UPDATE Passwords
SET lastUpdated = '2026-05-22'
WHERE lastUpdated < '2025-05-15';

-- Q9: How can the system completely delete a 'Windows 10' hardware endpoint profile
belonging to user 'Mustafa'?
DELETE FROM Devices
WHERE userID = (SELECT userID FROM Users WHERE username = 'Mustafa')
AND operatingSystem = 'Windows 10';
```

```
-- Q10: How can a user completely remove a compromised password record from their
dashboard using its account ID reference?
DELETE FROM Passwords
WHERE accountID = 10;

-- Q11: How can we dynamically evolve our device database schema by adding a new status
flag column for encryption tracking?
ALTER TABLE Devices ADD isEncrypted INT DEFAULT 1;

-- Q12: How can an administrator demonstrate safe environment management by creating a
temporary schema view and instantly dropping it?
-- Creating a view to instantly drop it for demonstration of schema management commands
GO
CREATE VIEW TemporaryTestingView AS SELECT username FROM Users;
GO
DROP VIEW TemporaryTestingView;
GO
```

-- Simple Queries commands

```
-- Q1: How can the system retrieve a complete directory of all available platform
categories arranged in alphabetical order?
SELECT categoryName
FROM Categories
ORDER BY categoryName ASC;

-- Q2: Which Windows devices logged into the system during the month of May 2025?
SELECT deviceName, userID, lastLogin
FROM Devices
WHERE operatingSystem LIKE 'Windows%'
AND lastLogin BETWEEN '2025-05-01' AND '2025-05-31 23:59:59';

-- Q3: What are the website name and website URL where category ID is 4?
SELECT websiteName, websiteURL
FROM Websites
WHERE categoryID IN (4);

-- Q4: How can we search note title and note content from Notes that contain
'Password', 'Key', or 'Backup'?
SELECT noteTitle, noteContent
FROM Notes
WHERE noteTitle LIKE '%Password%'
OR noteContent LIKE '%Key%'
OR noteContent LIKE '%Backup%';

-- Q5: Which users have a createdAt date after '2025-05-05'?
SELECT username, email, createdAt
FROM Users
WHERE createdAt > '2025-05-05';

-- Q6: How can we select logID, userID, and actionTime from AccessLogs where actionType
is 'PASSWORD_VIEW'?
SELECT logID, userID, actionTime
FROM AccessLogs
WHERE actionType = 'PASSWORD_VIEW';
```

```
-- Q7: Which accounts created but do not have a password record linked to them yet?
SELECT accountID, loginEmail, createdAt
FROM Accounts
WHERE accountID NOT IN (SELECT accountID FROM Passwords);
```

-- Complex Queries commands

```
-- Q1: How can we generate a comprehensive master manifest showing which websites and
specific login emails each system user is tracking?
```

```
SELECT U.username, W.websiteName, A.loginEmail, W.websiteURL
FROM Users U
JOIN Accounts A ON U.userID = A.userID
JOIN Websites W ON A.websiteID = W.websiteID;
```

```
-- Q2: How can we select websiteName and count accountID from Websites and Accounts
grouped by websiteName, sorted from highest to lowest?
```

```
SELECT W.websiteName, COUNT(A.accountID) AS TotalStoredAccounts
FROM Websites W
JOIN Accounts A ON W.websiteID = A.websiteID
GROUP BY W.websiteName
ORDER BY TotalStoredAccounts DESC;
```

```
-- Q3: How can we select userID and count logID from AccessLogs where actionType is
'PASSWORD_VIEW' or 'PASSWORD_UPDATE', having a count of 2 or more?
```

```
SELECT userID, COUNT(logID) AS SensitiveActionsCount
FROM AccessLogs
WHERE actionType IN ('PASSWORD_VIEW', 'PASSWORD_UPDATE')
GROUP BY userID
HAVING COUNT(logID) >= 2;
```

```
-- Q4: How can we use a WITH clause (CTE) to find userIDs whose total logID count is
greater than the average logs of all users?
```

```
WITH UserLogCount AS (
    SELECT userID, COUNT(logID) AS UserLogs
    FROM AccessLogs
    GROUP BY userID
)
SELECT SummaryTable.userID, SummaryTable.UserLogs
FROM (
    SELECT userID, UserLogs
    FROM UserLogCount
) AS SummaryTable
WHERE SummaryTable.UserLogs > (SELECT AVG(UserLogs) FROM UserLogCount);
```

```
-- Q5: How can we create a VIEW that uses INTERSECT to find loginEmails used in both
websiteID 3 and websiteID 5?
```

```
CREATE VIEW HighRiskBankingProfiles AS
SELECT loginEmail FROM Accounts WHERE websiteID = 3
INTERSECT
SELECT loginEmail FROM Accounts WHERE websiteID = 5;
GO
```

```
-- To run it:
SELECT * FROM HighRiskBankingProfiles;
```